

April 2026

OntoExtend: A Framework for Requirement-driven and Scalable Ontology Extension with LLMs

Anna Sofia LIPPOLIS^{* b,d} Mohammad Javad SAEEDIZADE^{* a} Stefan SCHMID^c
Simon BLATTNER^c Robin KESKISÄRKKÄ^a Aldo GANGEMI^b
Eva BLOMQVIST^a Andrea Giovanni NUZZOLESE^d

^a *Linköping University, Sweden*

^b *University of Bologna, Italy*

^c *Bosch, Germany*

^d *ISTC-CNR, Italy*

^{*}These authors contributed equally to this work.

Abstract. Ontology extension refers to the process of enriching an existing ontology in response to emerging requirements, making it more complete. This task is a resource-intensive and error-prone process. Large Language Models (LLMs) have shown promising performance on generating ontologies from scratch, but current approaches rarely tie ontology extension explicitly to requirements or reusable core models, and offer limited, systematic evaluation of LLM outputs. This paper introduces *OntoExtend*, a requirements-driven framework for ontology extension with LLMs. It uses retrieval-augmented generation (RAG) over relevant input ontologies and requirements in the form of competency questions to propose grounded extensions. We evaluate OntoExtend on 39 CQs from two use cases: a public EU-project ontology, Onto-DESIDE, and an industrial ontology from Bosch. The generated fragments show few structural issues, satisfy all functional evaluation tests, and are rated by ontology engineers as requiring minor to moderate revision before integration. These results suggest that OntoExtend is useful as a drafting assistant for requirement-driven ontology extension in real world scenarios, while remaining sensitive to CQ specificity and modelling profile.

Keywords. Ontology extension, Ontology generation, Ontology engineering, Large Language Models

1. Introduction

In recent years, Large Language Models (LLMs) have been increasingly integrated into various stages of the ontology engineering pipeline. Existing work includes ontology generation with LLMs [13,14,16,25], where an ontology is constructed from a set of requirements, and ontology evaluation [15,17,28], where the goal is to determine whether a given ontology correctly models those requirements. However, to the best of our knowledge, no prior study has examined how LLMs can extend an existing (input) ontology by reusing ontology elements while modelling additional requirements provided in the form

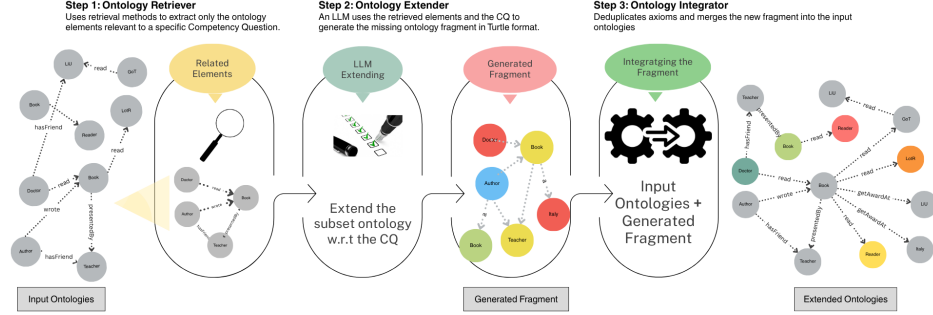


Figure 1. Overview of Ontoextend: 1) The *Ontology Retriever* extracts the relevant ontology elements of the input ontologies given a new competency question. 2) The *Ontology Extender* uses the retrieved ontology elements together with the competency question to prompt an LLM to generate missing ontology fragments. 3) The *Ontology Integrator* integrates the fragments into the extended ontologies.

of competency questions (CQs), i.e. natural language questions outlining and constraining the scope of an ontology. This task, which we call *ontology extension*, poses several challenges. For instance, input ontologies often contain hundreds of classes and properties, exceeding the input context limitations of current LLMs. Even when an ontology is small enough to fit within the large context window of state-of-the-art LLMs, they may be misguided by irrelevant details and produce off-target or inconsistent outputs [25].

Ontology extension is thus the systematic enrichment of an existing ontology to satisfy new functional requirements [22]. Using LLMs to produce these extensions could enable incremental maintenance of ontology modules as new requirements arise. To this end, we introduce the *OntoExtend* framework (Fig. 1), a retrieval-augmented approach for ontology extension that takes new CQs and the existing ontology as input. For each CQ, *OntoExtend* retrieves from the input ontology the named classes and properties, along with their axioms, that are relevant to the CQ. The CQ, together with this retrieved fragment, are then provided to an LLM as relevant context without exceeding its context window or overburdening it. In the rest of this paper, we use the term *fragment* to denote the self-contained set of RDF/Turtle axioms retrieved or generated for a specific extension step associated with a single CQ. This enables the LLM to generate highly contextualised ontology extensions that can be consistently integrated into the input ontology to model the new requirement.

The research questions (RQs) driving this work are: (i) Which embedding configuration is the most suitable for building a retrieval model that extracts a compact fragment of the ontology most relevant for a given CQ? (ii) Which LLMs are most effective at extending the retrieved fragment so that the CQ is correctly modelled? (iii) What evaluation criteria are required to assess the quality of the generated ontology fragments? (iv) What are the strengths and weaknesses of the extended ontologies produced by OntoExtend? To answer these questions, we propose the following contributions:

1. OntoExtend: our proposed framework that extends ontologies based on new requirements¹;

¹The code and the experiment data (ontologies and CQs), along with examples that could not fit in the paper, are available at <https://github.com/dersuchendee/OntoExtend>

2. Experiments comparing retrieval and prompting strategies to identify effective configurations for LLM-based ontology extension on two domain-specific, real-world case studies, demonstrating the approach’s applicability across different domains and requirements;
3. An evaluation methodology that assesses both structural validity and functional adequacy of generated extensions against the requirements in real-life settings through a user-based study.

The rest of the paper is organised as follows. Section 2 presents the related work; Section 3 describes OntoExtend; Section 4 describes the experimental setup; Section 5 shows the evaluation, and Section 6 reports the experimental results. Finally, Section 7 discusses the evaluation metrics, the impact and extensibility of the benchmark, and Section 8 concludes the paper and outlines future research directions.

2. Related Work

LLM-based ontology extension, at times called *ontology enrichment*, remains in an early phase with respect to other LLM-assisted knowledge engineering tasks, with approaches ranging from interactive tools to automated enrichment systems. Early approaches demonstrate the potential and limitations of semi-automated methods: Soares et al. [26] extended the Agricultural Product Types Ontology (APTO) using ChatGPT-4 for interactive generation of OWL taxonomic axioms. Similarly, Matieu and Groza [18] developed a Protégé plugin using a fine-tuned GPT-3 model to translate controlled natural language into OWL Functional Syntax. Work in Phrase2Onto [22], a prototype for ontology extension, showed promising results in this direction but was limited to toy ontologies. In addition to the mentioned limitations, these studies do not apply to larger ontologies.

Automated extension systems push toward greater autonomy while exposing new challenges: Taxoria [8] focuses on taxonomy enrichment by having LLMs propose child terms for existing nodes, which are then validated for semantic relevance before integration with provenance tracking, though control over hallucinated nodes and implicit requirement capture remain concerns. Wu et al. [29]’s online clustering framework extends this automation further, using LLM agents to propose and cluster new classes around existing concepts in an evolving medical ontology. Still in the biomedical domain, Dong et al. [3] focus on enriching OWL ontologies as formal KBs, finding that the baseline LLM-based methods are yet to achieve satisfying performance on the benchmark.

Comprehensive extension frameworks address broader ontology engineering needs: Kholmska et al. [12]’s work extends the input ontology through a multi-LLM workflow supporting concept search, extraction, alignment, and the generation of CQs and SPARQL queries, though this exposes LLM limitations in highly specialised domains and requires manual repair of shallow or incorrect suggestions. Joachimiak et al. [10] developed the Artificial Intelligence Ontology using an Ontology Development Kit workflow with AI-driven curation support. In García Fernandez et al. [7], during the LLM-based extension process, two important limits were found: LLMs hallucinated when asked about existing standards and reusable ontologies, and the LLM-generated extension was shallower than the manual gold standard. Human review stayed essential at every step.

Table 1. Comparison of the related works discussed in Section 2. Columns indicate whether a work explicitly reports ontology pitfall analysis (OOPS!), syntax or well-formedness checking (Syntax), semantic or logical consistency validation (Consist.), verification against requirements (Req. verif.), explicit presence of superfluous elements (Superfl. el), formal user evaluation (User eval.), expert-based assessment (Expert), evidence across multiple domains (Domain gen.), evaluation on an actively used ontology (Real-world Onto.), and evidence of scalability (Scal.). Symbols: Y = yes, P = partial, N = not reported.

Ref.	Approach	Structural			Functional		User eval.	Expert	Domain gen.	Real-world Onto.	Scal.
		OOPS!	Syntax	Consist.	Req. verif.	Superfl. el.					
[26]	Interactive taxonomy extension	N	N	N	N	N	Y	Y	Y	Y	N
[18]	Protégé plugin / CNL to OWL	N	N	P	N	N	N	N	N	N	N
[22]	Prototype ontology extension	N	N	N	N	N	Y	Y	Y	Y	N
[8]	Taxonomy enrichment (Taxoria)	N	N	N	N	N	N	N	N	Y	P
[29]	Online clustering framework	N	N	N	N	N	N	N	N	Y	Y
[3]	Biomedical enrichment benchmark	N	N	N	N	N	N	N	N	Y	P
[12]	Multi-LLM extension workflow	N	P	P	N	N	N	N	N	Y	P
[10]	AI Ontology curation support	N	P	Y	N	N	N	N	N	Y	P
[7]	Human-reviewed ontology extension	N	Y	N	Y	N	N	N	Y	Y	N
[9]	RAG ontology construction	N	N	N	N	N	N	N	N	N	Y
[1]	Research ontology construction	N	N	N	N	N	N	N	N	Y	Y
[17]	Ontology generation	N	N	N	N	N	N	N	N	Y	N
[21]	Ontology-Toolkit	N	N	N	N	N	N	N	N	N	N
[4]	Ontology generation from seeds/corpora	N	N	Y	N	N	N	N	N	N	N
[27]	Ontology transformation support	N	N	N	Y	N	N	Y	N	Y	N
[25]	Ontology engineering assistant	N	Y	N	Y	N	N	N	Y	N	N
[16]	Ontology engineering assistant	Y	Y	N	Y	Y	Y	N	Y	Y	N
Ours	OntoExtend	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y

These works reveal that current LLM-based ontology extension is predominantly semi-automatic enrichment of existing inputs with a bias toward taxonomic growth, facing recurring constraints around reproducibility, reliance on interactive tools, limited support for complex axioms, and continued necessity of human judgment for requirement elicitation and validation. However, none of these methods focuses on the retrieval of existing ontology elements from baseline ontologies, while our approach integrates retrieval mechanisms directly into the extension process. Furthermore, most of the works use minimal to no formal evaluation.

Several related works sit nearby but are less about extending a mature input ontology and more about generating or reconstructing ontologies from scratch [1,4,9,17,21] A parallel line of work looks at ontology transformation and general LLM-based assistance for ontology engineering [16,24,25,27]. These works reinforce the usefulness of LLM support throughout the ontology lifecycle, but they do not yet address retrieval-aware, requirement-driven extension of a mature input ontology.

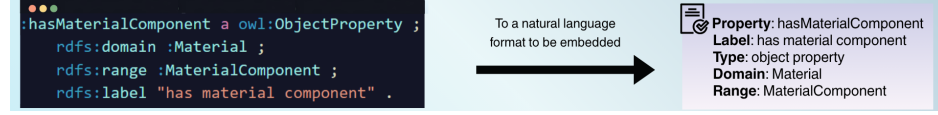


Figure 2. Example of the element to be embedded.

3. The OntoExtend framework

The proposed system implements a retrieval-based pipeline for ontology extension, organised into three principal subsystems: the Ontology Retriever, the Ontology Extender, and the Ontology Integrator, illustrated in Fig. 1. Throughout this paper, input ontologies and reference ontologies refer to the same artefacts: the ontology (or ontologies) the user wants to extend, which are also indexed by the Ontology Retriever for element retrieval. The RAG component indexes the same ontology files that constitute the extension target, so that generated fragments are grounded in and consistent with the existing modelling choices. Optionally, previously generated fragments can be re-indexed so that later CQs build on earlier ones.

3.1. Ontology Retriever

Since the whole set of input ontologies may not fit into the context size of current LLMs, the Ontology Retriever is responsible for constructing and querying a semantic index over the input ontologies (i.e., the ontologies to be extended).

At preprocessing time, the retriever component parses each ontology file specified as input by the user and iterates over all declared OWL entities, including classes, object properties, data properties, annotations, and their SHACL shapes if available. For each entity, it constructs an *OntologyElement* record containing: the entity IRI; human-readable labels and comments if present; domain and range declarations (where applicable); super- and sub-class (or sub-property) relations; and a verbatim Turtle snippet capturing the canonical declaration of the entity.

These *OntologyElements* form the input knowledge base used for subsequent retrieval. To support semantic search, each *OntologyElement* is embedded into a dense vector representation using a configurable sentence embedding model. Each element is serialised as a pipe-delimited string combining its URI local name, human-readable label, comment, element type, and domain/range before embedding, e.g., ‘hasMaterialComponent | has material component | ... | Type: object property | Domain: Material | Range: MaterialComponent’. The resulting vectors are normalized and stored in a FAISS index [5] configured for inner-product similarity. This choice enables efficient nearest-neighbour search over large ontology collections while preserving cosine-like similarity semantics. An example of the element to be embedded with the pipe format is shown in Figure 2.

At query time, a user-supplied CQ is embedded into the same vector space and used to interrogate the FAISS index. The retriever returns the top-k (the default is 20) most similar *OntologyElement* instances, effectively selecting those ontology entities whose textual and structural descriptions are most relevant to the CQ. This retrieval step ensures that the subsequent generation is tightly anchored in the terminology and modelling patterns of the reference ontologies. Moreover, it is the key for promoting reuse of existing ontology elements.

3.2. *Ontology Extender*

The Ontology Extender assembles the final prompt for the LLM with the task to propose consistent extensions. Retrieved elements are grouped by their source ontology and rendered as Turtle snippets, which are injected into the prompt as read-only context. In addition, it constructs a unified prefix block that merges the namespace declarations of all input ontologies, ensuring that every element IRI retrieved from any source ontology resolves correctly in the generated output and can be directly reused by the LLM without namespace conflicts. OntoExtend also enables ontology engineers to configure different prompts so that users can select among the most suitable templates at run-time.

Before a fragment is accepted, it is passed through a two-stage validator: a Turtle parser verifies syntactic correctness, and a constraint checker verifies whether generated properties satisfy the modelling conventions required in the selected use case. In the two profiles used in this evaluation, this includes explicit ‘`rdfs:domain`’ and ‘`rdfs:range`’ declarations, but this is a configurable governance constraint and not a universal criterion for ontology correctness. Fragments failing validation are either retried or flagged.

Given the assembled prompt, this Ontology Extender calls the LLM service and extracts the returned Turtle code block, which is treated as a candidate TBox fragment.

Prompt variants and deployment profiles: To support heterogeneous deployment requirements, the system maintains different prompt templates that configure the extender stage. These prompts are cleanly decoupled from the input retrieval, generation, and integration pipeline.

In our work, we used two distinct templates according to the domain of the data we experimented with (see Section 4). For the industrial ontology, we develop a SHACL-based prompt template, whereas for the EU ontology we defined a prompt template for the generic use of restrictions and reuse of classes policy. The former instructs the LLM to output SHACL NodeShape and PropertyShape definitions following specific naming conventions and including `sh:name` labels; this is essential for downstream compliance workflows that consume SHACL artefacts. Its modelling guidance largely stops after requiring appropriate domain and range declarations. In contrast, the latter prompt template requires that every newly introduced class and property be annotated with both `rdfs:label` and `rdfs:comment`. Both impose strong reuse constraints, instructing the LLM to reuse existing ontology elements without redeclaring it. By externalising these prompt configurations, the system can switch between companies and use cases without any changes to the underlying system components.

3.3. *Ontology Integrator*

The Ontology Integrator module is responsible for incorporating the generated fragment into the input ontologies. Its operation is limited to deduplication (removing repeated axioms, named classes, properties, and prefixes) followed by concatenation of the cleaned fragment with the input ontologies.

The Ontology Integrator additionally forwards each generated fragment to the Ontology Retriever for indexing, thereby reducing the likelihood that the Ontology Extender would recreate similar elements (classes or properties) when subsequent CQs were processed. Re-indexing allows each generated fragment to serve as a reference for subsequent CQs, promoting cross-fragment consistency. However, this was disabled in our evaluation so that each CQ fragment could be assessed independently.

4. Experimental setup

In this section, we describe the experimental setup, including the dataset we created, parameter tuning for the retrieval and the extender module, and the evaluation criteria.

Dataset Creation: The evaluation covers two settings: four modules of the Onto-DESIDE ontology network², treated jointly as one setting (20 CQs), and an internal Bosch ontology (19 CQs). Onto-DESIDE addresses circular economy data interoperability, whereas the Bosch setting follows a more manufacturing industrial profile. The dataset was manually created by systematically removing selected classes and their related properties from each ontology, and then formulating CQs about the removed elements. The procedure is outlined below:

1. Select a random set of classes C from the ontology.
2. For each class c , remove c and add to the set P all properties whose domain or range restrictions references c .
3. Iteratively expand the removal process by adding c all subclasses of each removed class to C , and include in P any properties that reference these subclasses. This step is repeated iteratively until no additional subclasses are found.
4. Finally, construct a set of CQs associated with C and P . Each CQ is formulated by asking what each corresponding class with its properties, were intended to represent or accomplish within the original ontology.

This procedure yields a set of CQs that systematically probe the missing ontology components, enabling an effective evaluation of the OntoExtend framework.

Dataset statistics: The EU-project parts have 20 CQs in total, while the Industry ontology comprises 19 CQs. Detailed counts of tokens, the sum of classes and properties, and axiom counts for each subset are provided in the GitHub repository. The input ontologies were converted to Turtle with compact prefix declarations to reduce size. Token counts are measured using the GPT-4o tokeniser (<https://platform.openai.com/tokenizer>), see Table 2.

Table 2. Token counts of input ontologies after conversion to Turtle.

Ontology fragment	Size (tokens)	Axioms	Classes+Properties
EU-project — part 1	75 000	2920	405
EU-project — part 2	22 000	958	242
EU-project — part 3	25 000	1172	270
EU-project — part 4	6 000	228	54
Industry use case	22 000	979	134

Parameter and configuration tuning

We ran the experiment on a separate subset to find out the best embedding model and parameter configuration to test on the main experiment, along with the best prompt to minimise the common mistakes by LLMs in ontology generation.

Best retrieval configuration

To embed each ontology element into a vector with good settings, we first ran a small experiment to choose the embedding model and text configuration before the main study

²<https://cordis.europa.eu/project/id/101058682>

to find a good set of parameters and embedders. Basing on the results of previous research [14,25], we compared three OpenAI embedding models: `text-embedding-3-small`, `text-embedding-3-large`, and `text-embedding-ada-002` under four different styles of formulating ontology elements as a raw text before sending them into embedding mode. In particular, we varied the delimiter used to separate consecutive elements, comparing the use of a pipe character (`|`) with the use of a newline character (`\n`). For each configuration, we manually judged relevance on five CQs not present in the test data and computed precision at 3 and 20 (focusing on top-3 retrieved for answer quality in case of creating a small ontology subset, but still monitoring top-20 to have an estimate of recall). In particular, we computed precision at cutoffs 3 and 20 for the retrieved elements being tagged as similar to the input CQ by the Ontology Retriever. Evaluations are done manually by two ontology engineers cross cross-checking each other’s work, letting P_3 and P_{20} denote precision at 3 and 20 (often referred to as $P@3$ and $P@20$). Our primary selection metric is a weighted average $M_w = 0.7 P_3 + 0.3 P_{20}$ which emphasises the quality of the very top-ranked results (those most likely to be consumed by the LLM in Ontology Extender) while still rewarding configurations that retrieve more relevant items in the top 20. As an additional check, we also monitored the multiplicative average of them $M_{\text{prod}} = P_3 \cdot P_{20}$, which penalises configurations that perform well at only one cutoff. Both metrics agreed on the same best configuration per model.

Across all 12 configurations, the best overall result came from `text-embedding-ada-002` with pipe-separated element axioms and comments included. On this basis, we selected that as the embedding model for the main experiment. Inputs of the top configurations are shown in Table 3.

Table 3. Embedding model comparison results by examining whether to include comments in the embedding and how to separate axioms in the raw text.

Embedding model	Separator, Comments	M_{prod}	M_w
<code>text-embedding-3-small</code>	Newline, with Comments	0.22	0.62
<code>text-embedding-3-large</code>	Newline, without Comments	0.20	0.54
<code>text-embedding-ada-002</code>	Pipe, with Comments	0.23	0.63

Best prompt and LLM extender

For the LLMs, we selected `o1-preview`, reported as the best-performing model in previous work [16], and `GPT-5`, the latest LLM from OpenAI. We did not select other models or families because previous studies reported lower performance relative to `o1-preview` for what concerns ontology generation tasks [16].

Prompt development followed the same tuning procedure used for retrieval. After fixing the retrieval configuration and integrating it into the extension framework, we evaluated several prompt variants on a small development set of CQs (disjoint from the main evaluation set) to identify formulations that minimised typical LLM errors in ontology construction. The initial prompt was modelled closely on the prompt used in previous work [16] and was augmented with an explicit list of observed pitfalls not to be included in the output. This baseline was refined iteratively: after each pilot run, two ontology engineers inspected generated axioms for recurrent problems and adjusted the prompt wording, constraints accordingly.

The final prompt comprises (i) a general template that specifies the task and required output format, (ii) a section into which the Ontology Retriever’s elements are inserted,

(iii) a placeholder for the given CQ, and (iv) a directive that indicates the expected modelling formalism. The framework populates all components automatically, while the style directive may be supplied by the user to constrain output style. For instance, in the EU-Project, this section requested OWL restrictions for creating restrictions in the generated ontologies; in the industry use case, we disallowed OWL restrictions and required only SHACL shapes. Consequently, the two use cases should not be interpreted as testing the same level of OWL expressivity. The EU-project setting evaluates OWL-style fragment generation, whereas the industry setting evaluates extension under a SHACL-oriented engineering profile used in that deployment context. The final prompt template is available on the Github repository.

4.1. Evaluation criteria

Building on previous studies in LLM-assisted knowledge engineering [16], we adopt a multidimensional evaluation setup that combines structural and functional metrics along with human evaluation.

Structural Metrics. We employ the Ontology Pitfall Scanner (OOPS!) [23] for reporting on pitfalls, Pellet reasoner and syntax checking through the RDFLib Python library³.

Functional Metrics. To assess the generated ontologies, we employ CQ verification [2] and counting of superfluous elements based on Lippolis et al. [16] by modifying the definition of the latter for the specific task of ontology extension. CQ verification evaluates whether a given CQ is actually represented in the ontology by attempting to formulate a SPARQL query that retrieves an answer for that CQ. If no such SPARQL query can be written to obtain an answer, the CQ is considered *not modelled* in the ontology. Likewise, [16] defines a superfluous element in a way that treats some generated classes and properties as superfluous because they cannot be referenced directly in the SPARQL query used for CQ verification. In our work, we adopt a more specific definition: a *superfluous element* is a named class or property that: 1) is not mentioned in the verification SPARQL used to in CQ verification, and 2) is not connected to any component appearing in that query by a `subClassOf` or `subPropertyOf` relation. Under this definition, elements that are absent from the verification query but are hierarchical relatives (subclasses or subproperties) of query components are *not* considered superfluous, while genuinely disconnected named classes and object properties are flagged as superfluous.

Survey. Finally, we created a survey to ask six ontology engineers from both academia and industry to evaluate the extended ontologies. We adopted evaluation criteria adapted from Monka et al. [19], where two dimensions for evaluating *SPARQL* generation are proposed: *Correctness* and *Completeness*. In our work, *Correctness* measures the syntactic and semantic quality of the generated fragment in isolation, and *Completeness* measures the sufficiency of the generated fragment after it has been combined with the input ontology; it captures the amount of effort required by an ontology engineer to make the fragment usable. Users rated the generated ontology fragments on a five-point Likert scale along two dimensions, namely *Correctness* and *Completeness*.

For *Correctness*, a rating of 1 indicates a generation failure, for example because the output exceeded token limits. A rating of 2 denotes a syntactically erroneous fragment, such as one containing issues in the generated code. A rating of 3 is assigned when

³<https://rdflib.readthedocs.io/en/stable/>

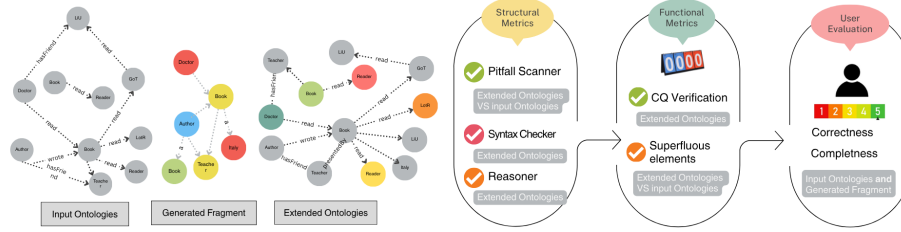


Figure 3. Evaluation workflow: input ontologies, the generated fragments, and the extended ontologies are fed to different evaluation metrics to create the final report.

the fragment is syntactically correct but semantically incorrect, for instance due to an inappropriate taxonomy. A rating of 4 corresponds to a fragment that is both syntactically and semantically correct, but does not fully satisfy the intent of the competency question (CQ). Finally, a rating of 5 indicates a fragment that is syntactically and semantically correct and fully aligned with the intended meaning of the CQ.

For *Completeness*, a rating of 1 means that the generated ontology fragment is not useful and would require complete manual reworking. A rating of 2 indicates that significant changes are needed, corresponding roughly to 25–40% of the fragment requiring modification. A rating of 3 reflects the need for moderate changes, with approximately 11–25% of the fragment needing revision. A rating of 4 denotes that only minor changes are required, typically affecting around 1–10% of the fragment. A rating of 5 indicates that the generated fragment, in combination with the input ontology, is complete and correct.

5. Evaluation

In this section, we present the methodology used to evaluate the generated ontology extensions based on the criteria introduced in Section 4 in detail.

5.1. Structural Evaluation

As shown on the right side of Fig. 3, each generated extension module is validated for correct Turtle syntax via the RDFLib Python library. To assess common modelling pitfalls, we run OOPS! on the input ontology both before and after integrating the generated extension, compare the reported pitfalls, and document any new issues introduced by the extension. In addition, we execute the Pellet reasoner to perform a consistency check on the input ontology after the generated extension has been integrated.

5.2. Functional Evaluation

Functional adequacy (CQ verification). As described in Section 4.1, we perform CQ verification on each generated extension after it is added to the input ontology to ensure that each CQ is correctly modelled (see Figure 3). Two ontology engineers independently verify the CQ modelling decisions and then cross-check each other’s annotations; any disagreement is resolved through discussion. This adjudication process follows the same general workflow as in the work by [14].

Superfluous elements. After CQ verification, the same pair of engineers inspects the integrated file to identify and count superfluous elements introduced by the generated extension by comparing the number of superfluous elements before and after adding the extension. As detailed in Section 4.1, each decision is cross-checked, and we report the number of superfluous elements that were generated but deemed unnecessary by the OntoExtend framework.

5.3. Evaluation by Ontology Engineers

In the experiment, we recruited six ontology engineers from both industry and academia to evaluate the generated extension modules. After a short briefing that explained the task and the survey form, each evaluator received a set of CQs, the input ontologies, and the generated ontology fragments for each CQ. For every generated extension, the evaluators answered the two survey questions regarding correctness and completeness (see section 4.1). The evaluators were also encouraged to provide free-text comments. Finally, the evaluators also participated in a debriefing session to share general feedback. Participants were free to use any tools they preferred to visualise the ontology artefacts (e.g., Protégé, Topbraid EDG, VSCode).

After data collection, we computed the mean correctness and completeness metrics from the surveys. We also computed the *Fleiss* weighted observed agreement P_o [20] to quantify the inter-annotator agreement among our evaluators.

6. Results

In this section, we present the results of our experimental setup according to the evaluation criteria outlined in Section 5 of the paper.

6.1. Results of Structural Evaluation

Firstly, the generated ontology fragments almost did not exhibit any syntax issues in Turtle. Second, the analysis of the integrated ontologies with OOPS! shows that the generated extension fragments do not introduce any new *critical* or *important* modelling pitfalls after being integrated into their input; only a small number of minor issues were observed in specific cases. For the EU-project ontology, we found two types of minor issues. First, P02 (creating synonyms as classes) appears in a single EU-project only: in that case, each generated extension contributed one P02 instance. Second, P04 (unconnected ontology elements) was detected in just two of the four EU-project use cases; in those two cases, each extension added at most one additional P04 instance.

For the industry ontology, OOPS! reported P08 (missing annotations). This behaviour is likely due to the annotation-free style of the input ontology: OntoExtend appears to have replicated that style as it is asked in the prompt, yielding missing annotation warnings when annotations were expected by OOPS!. On average, P08 occurs approximately 3.7 times per CQ-based extension for both LLMs.

Comparing OntoExtend to existing work on ontology generation [6,13,16], which showed a considerable number of *important* and *critical* OOPS! pitfalls, OntoExtend exhibits only a few minor pitfalls. Overall, the results indicate that the generated extensions are structurally acceptable and do not systematically introduce new modelling defects;

Table 4. Summary of structural (syntax and OOPS!) and functional (CQ-verification and percentage of superfluous elements) evaluation for the EU-Project and the Industry ontologies, all LLMs combined and separated.

Use case	OOPS! P2,P4&P8 (o1-prev,GPT-5)	CQ-verification (o1-prev,GPT-5)	Syntax errors (o1-prev,GPT-5)	Superfluous elements (o1,GPT-5)
EU-Project	13 (7,6)	100% (100%, 100%)	0% (0%,0%)	2% (3.8%,0%)
Industry	141(70,71)	100% (100%,100%)	2.5% (5%,0%)	0% (0%,0%)

Table 5. Survey results on evaluating ontology extension fragments for the EU-Project and industry ontologies. The ratings are between 1 and 5 (see section 4.1)

Model	Metric	Industry		EU-Project	
		Mean	Fleiss P_o	Mean	Fleiss P_o
o1-preview	Correctness	4.91	0.97	3.69	0.80
o1-preview	Completeness	4.56	0.89	3.11	0.85
GPT-5	Correctness	4.96	0.98	3.66	0.87
GPT-5	Completeness	4.54	0.87	2.94	0.87

the limited minor issues observed are confined to specific use cases and are straightforward to fix.

6.2. Results of Functional Evaluation

The results of CQ verification show that all CQs in both use cases are correctly modelled, with no minor issues of the type identified by Saeedizade and Blomqvist [25]. Both GPT-5 and o1-preview produce extension fragments with only negligible amounts of superfluous elements. A comparison of the number of superfluous elements with the work in [16] further highlights this improvement. Using our refined definition of superfluous components, the ontology fragments generated in this work contain fewer than 2% unnecessary elements, whereas [16] reported around 30% superfluous elements in their generated ontologies (35% with [16]’s original definition of superfluous elements).

6.3. Results of survey from Engineers’ Evaluation

Six ontology engineers, comprising three engineers for the EU-Project and three who worked on the industry ontology, evaluated the generated ontology fragments by filling out the survey, results shown in Table 5.

Engineers who evaluated fragments of the EU ontologies identified some systematic deficiencies. First, they observed unconnected elements within fragments that ought to have been linked to the input ontologies, e.g. via a `subClassOf` relation. Second, class naming was often suboptimal (for example, excessively specific names), indicating recurrent poor modelling patterns. Model-specific issues were also reported. Outputs produced by GPT-5 frequently omitted explicit domain and range declarations for newly introduced properties, imposed incorrect semantic restrictions (e.g., inappropriate use of `allValuesFrom` or `complementOf`), and employed overly simplified names. Conversely, fragments generated by o1-preview commonly redefined existing classes, contained syntactically incorrect axioms, which were not shown by the tools used for the Structural evaluation, assigned incorrect domains to some properties, and created object

properties that appeared too specific compared to individual CQs. Overall, the evaluators with high agreement concluded that the generated fragments require moderate revision before they can be safely integrated into the input ontologies.

Regarding the industry ontology, the main errors concerned that in few cases, only SHACL property shapes were added without the corresponding object property plus proper `rdfs:domain` and `rdfs:range` definitions, and in a few cases the `sh:datatype` was missing. Furthermore, it was noted that sometimes LLMs omitted the required `rdfs:subClassOf` axioms and lacked comment annotations for classes and properties. This is valid for both models. Specific only to `o1-preview`, it tends to introduce many additional Property shapes and object properties that are not needed to answer the CQs. There is also concern in two comments that extensions were generated without grounding from the CQ or the input ontologies. In this case, the LLM anticipated some extensions purely based on patterns available in the input ontology. Overall, however, the results regarding the industry ontology show a high degree of user satisfaction (minor or no changes needed to the fragments) and strong observed agreement among the evaluators.

7. Discussion

In this section, we analyse the results, outline the main limitations of our study and experimental setup, and discuss directions for future work.

Overall results: Overall, our results show that OntoExtend can reliably produce high-quality ontology extensions across both domains we have considered. The generated fragments are almost always syntactically valid, introduce only a small number of minor OOPS! pitfalls, especially when compared with existing related work. They correctly model all evaluated CQs, and contain fewer than 2% superfluous components, while the six ontology engineers generally judged them to be syntactically and semantically adequate and requiring at most minor to moderate edits. Taken together, these findings indicate that OntoExtend already offers a practically useful level of automation for real-world ontology extension and maintenance. In practice, OntoExtend functions as a drafting assistant: it generates a complete extension module that users typically only need to review and lightly edit. This keeps human-tool interaction minimal, shifting effort from manual axiom authoring to quick validation and targeted refinement. This significantly lowers the barrier to entry for domain experts, allowing them to expand ontologies with less reliance on often lacking ontology engineers.

Formulation and quality of the CQs affecting the results: Industry ontology engineers assessed the industry ontology and the EU-project ontologies (the latter developed by academic partners), and inspected the generated extension for the EU-project. Respondents were encouraged to add free-text notes on each generated module.

Our evaluation covered two distinct usage scenarios: (i) extending an ontology from highly specific, pre-determined CQs (the tool acts as an assistant for writing axioms), and (ii) constructing or extending ontologies from more general, open CQs (the tool must make substantive modelling choices). Although the same overall procedure was used to create the datasets, the EU-project CQs differ qualitatively from the industry CQs. The latter were deliberately very focused and targeted: they were authored jointly by domain experts and ontology engineers so that high-level modelling choices were already fixed and each CQ primarily required the tool to generate a missing class or a small set of properties from the natural-language question. By contrast, the CQs for the EU-Project

ontology were defined in a more open manner. I.e., the authors, mostly domain experts, left many of the modelling decisions to the tool. Consequently, the tool in this case has to make the open modelling decisions when generating the ontology fragments.

This difference in CQ style affects the evaluation. For the EU-project CQs, the LLMs often produced reasonably structured extensions, but ontology engineers judged some outputs as less complete because additional manual refinement was needed to align the results with modelling choices implied by the full specification. Thus, lower correctness and completeness scores in this setting reflect rather the limitations of the CQ definition (i.e., missing clarity on the expectations and contextual detail), not necessarily a weakness of the OntoExtend framework. In the industry setting, the tool mainly needed to render valid axioms according to an input ontology and hence achieved much higher scores for completeness and correctness. The lower scores on correctness and completeness observed in the EU-project scenario from the evaluators' survey results can therefore be explained as a result of the different task complexity of the ontology generations.

LLM behaviour as an implicit quality measure: As discussed previously, the same framework is applied to two qualitatively different CQ sets and associated ontologies: one in which CQs are tightly scoped, and another in which CQs lack clear expectations and context, leaving many modelling decisions open. Our analysis shows that OntoExtend tends to perform better in terms of correctness and completeness, alignment with expert expectations, and reduced need for post-editing when the CQs and background ontologies are well structured, precise, and internally coherent. These findings in turn suggest that LLM performance can serve as a proxy indicator for the quality of requirements (CQs) and/or ontological artefacts: if a given set of CQs and input ontologies systematically yields higher quality extensions with fewer modelling errors, then this combination is in a pragmatic sense “better” for downstream ontology engineering with LLMs (and probably also humans). Conversely, when the same solution struggles, this most likely is a result of ambiguity, under-specification, or hidden modelling assumptions in the input. In line with recent discussions on using LLMs as auxiliary evaluators in knowledge engineering, our results therefore support the interpretation of LLM behaviour as a novel, tool-centric dimension of ontology and requirement quality.

Practical impact: Sending very large ontology fragments (e.g., ~75k-token) directly to an LLM resulted in substantially longer end-to-end response times and higher API costs in our experiments. By contrast, OntoExtend retrieves a compact subset of the ontology and provides only that subset to the LLM; this greatly reduces the typical LLM turnaround and lowers the amount of data transmitted and the associated cost.

Limitations and future work: Despite the promising aspects of this work, it also has limitations. First, our evaluation is restricted to two domain-specific use cases; a broader set of domains and modelling scenarios would be necessary to draw more general conclusions. Second, we experimented with only a small number of LLM configurations, and a more systematic comparison across different models and providers could reveal important performance differences. A further potential threat to the validity of our results is data leakage, i.e., the possibility that the underlying LLMs have been exposed during pre-training to ontologies, CQs, or related documentation that are similar to (or overlap with) our experimental material. In our setup, for the EU-Project ontology, the ontologies have been publicly released, but the material concerning CQs and their coverage is not publicly available. For the industry ontology, it was not disseminated in public code or data repositories, which reduces the risk that related artefacts have been

seen during model training. Future work will explore more rigorous safeguards against leakage, such as on-the-fly benchmark generation, experiments with open-weight models trained on controlled corpora, and dedicated checks for overlap between evaluation data and known pre-training sources. Finally, our findings indicate that the quality and formulation of requirements (in our case, CQs) have a substantial impact on the outcome. Future work should explicitly analyse and improve CQ quality itself, for example by (semi-)automatically generating CQs, or mapping them to existing CQ templates (see the work by Keet et al. [11]).

8. Conclusion

In this work, we introduced OntoExtend, a scalable requirement-driven framework that extends one or more input ontologies according to the requirements specified in a set of CQs. OntoExtend processes each CQ individually and extracts, for each CQ, the relevant existing ontology elements as context for an LLM to generate the missing ontological structures necessary to fully answer the CQ in a coherent manner. To answer our research questions, we found that using `text-embedding-ada-002` as an embedder produced the most effective retrieval subsets for our experiments (RQ1). Furthermore, both `GPT-5` and `o1-preview` yielded comparable performance when supplied with the retrieved context (RQ2): the generated fragments contained almost no Turtle syntax errors, did not introduce any new *critical* or *important* OOPS! pitfalls, only a negligible amount of minor pitfalls, and exhibited only a small fraction of superfluous elements. We propose a multi-faceted evaluation (RQ3), combining syntactic checks, OOPS! analysis, CQ verification, measurement of unnecessary classes/properties (superfluous), and an analysis of the generated outputs by six ontology engineers. This evaluation setup shows that OntoExtend is able to extend ontologies that are both correct and complete according to ontology engineers. Overall, for what concerns the benefits and weaknesses of the extended ontologies produced by OntoExtend (RQ4), the extended fragments are structurally and syntactically correct, accurately model all evaluated CQs, and add fewer than 2% superfluous elements, with engineers often needing only minor edits in the industry use case. The positive evaluation from ontology engineers shows high potential for the usage of OntoExtend in real scenarios. However, we still observe weaknesses such as occasional missing domain/range declarations, suboptimal or overly specific names, and some unconnected elements. Moreover, performance degrades when CQs are open-ended or underspecified, showing that OntoExtend is effective when requirements are precise but sensitive to the quality of the input CQs.

Acknowledgements. This project has received funding from the European Union’s Horizon Europe research and innovation programme under grant agreements no. 101058682 (Onto-DESIDE) and the Swedish Vinnova-funded project SwePass (Dnr. 2024-02504). This work was also supported by the PhD scholarship “Discovery, Formalisation and Re-use of Knowledge Patterns and Graphs for the Science of Science”, funded by CNR-ISTC through the WHOW project (EU CEF programme - grant agreement no. INEA/CEF/ICT/A2019/2063229) and COST Action CA23147 GOBLIN – Global Network on Large-Scale, Cross-domain and Multilingual Open Knowledge Graphs, supported by COST (European Cooperation in Science and Technology, <https://www.cost.eu>).

Use of Generative AI. ChatGPT was used to enhance the readability of some of the text and improve the language of this paper, after the content was first added manually. After using this service, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

- [1] Aggarwal, T., Salatino, A., Osborne, F., Motta, E.: Leveraging large language models for generating research topic ontologies: A multi-disciplinary study. *arXiv preprint arXiv:2508.20693* (2025)
- [2] Blomqvist, E., Seil Sepour, A., Presutti, V.: Ontology testing-methodology and tool. In: *International Conference on Knowledge Engineering and Knowledge Management*. pp. 216–226. Springer (2012)
- [3] Dong, H., Chen, J., He, Y., Horrocks, I.: Ontology enrichment from texts: A biomedical dataset for concept discovery and placement. In: *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. pp. 5316–5320 (2023)
- [4] Doumanas, D., Soularidis, A., Spiliotopoulos, D., Vassilakis, C., Kotis, K.: Fine-tuning large language models for ontology engineering: A comparative analysis of gpt-4 and mistral. *Applied Sciences* **15**(4), 2146 (2025)
- [5] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.E., Lomeli, M., Hosseini, L., Jégou, H.: *The faiss library* (2024)
- [6] Fathallah, N., Das, A., Giorgis, S.D., Poltronieri, A., Haase, P., Kovriguina, L.: Neon-gpt: a large language model-powered pipeline for ontology learning. In: *European Semantic Web Conference*. pp. 36–50. Springer (2024)
- [7] García-Fernández, J., Verhoosel, J., Ubacht, J., Bakker, R.M.: Ontology engineering with large language models: Unveiling the potential of human-llm collaboration in the ontology extension process. *extraction* **7**, 15 (2025)
- [8] Ghamlouch, Z., Alam, M.: Enriching taxonomies using large language models. In: *ECAI 2025-28th European Conference on Artificial Intelligence (Demo Track)* (2025)
- [9] Huang, Y., Karabulut, E., Degeler, V.: Large language model for ontology learning in drinking water distribution network domain (2024)
- [10] Joachimiak, M.P., Miller, M.A., Caufield, J.H., Ly, R., Harris, N.L., Tritt, A., Mungall, C.J., Bouchard, K.E.: The artificial intelligence ontology: Llm-assisted construction of ai concept hierarchies. *Applied Ontology* **19**(4), 408–418 (2024)
- [11] Keet, C.M., Mahlaza, Z., Antia, M.J.: Claro: a controlled language for authoring competency questions. In: *Research Conference on Metadata and Semantics Research*. pp. 3–15. Springer (2019)
- [12] Kholmska, G., Kenda, K., Rozanec, J.: Enhancing ontology engineering with llms: From search to active learning extensions. *Proceedings of Data Mining and Data Warehouses–Sikdd* (2024)
- [13] Lippolis, A.S., Ceriani, M., Zuppiroli, S., Nuzzolese, A.G.: Ontogenia: Ontology generation with metacognitive prompting in large language models. In: *European Semantic Web Conference*. pp. 259–265. Springer (2024)
- [14] Lippolis, A.S., Saeedizade, M.J., Keskisarkka, R., Gangemi, A., Blomqvist, E., Nuzzolese, A.G.: Assessing the capability of large language models for domain-specific ontology generation. *ELMKE workshop* (2025)
- [15] Lippolis, A.S., Saeedizade, M.J., Keskisärkkä, R., Gangemi, A., Blomqvist, E., Nuzzolese, A.G.: Large language models assisting ontology evaluation. In: *International Semantic Web Conference*. pp. 502–520. Springer (2025)
- [16] Lippolis, A.S., Saeedizade, M.J., Keskisärkkä, R., Zuppiroli, S., Ceriani, M., Gangemi, A., Blomqvist, E., Nuzzolese, A.G.: Ontology generation using large language models. In: *European Semantic Web Conference*. pp. 321–341. Springer (2025)
- [17] Llugiqi, M., Ekaputra, F.J., Sabou, M.: From experts to llms: Evaluating the quality of automatically generated ontologies. In: *2nd Workshop on Evaluation of Language Models in Knowledge Engineering (ELMKE)*, co-located with ESWC-25, to appear (2025)
- [18] Mateiu, P., Groza, A.: Ontology engineering with large language models. In: *2023 25th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*. pp. 226–229. IEEE (2023)
- [19] Monka, S., Grangel-González, I., Schmid, S., Halilaj, L., Rickart, M., Rudolph, O., Dias, R.: Enhancing manufacturing knowledge access with llms and context-aware prompting. *arXiv preprint arXiv:2507.22619* (2025)
- [20] Moons, F., Vandervieren, E.: Measuring agreement among several raters classifying subjects into one or more (hierarchical) categories: A generalization of fleiss’ kappa. *Behavior research methods* **57**(10), 287 (2025), <https://doi.org/10.3758/s13428-025-02746-8>
- [21] Plu, J., Escobar, O.M., Trouilleux, E., Gapin, A., Troncy, R.: A comprehensive benchmark for evaluating llm-generated ontologies. In: *The Semantic Web-ISWC* (2024)

- [22] Pour, M.A.N., Li, H., Armiento, R., Lambrix, P.: Phrase2onto: a tool to support ontology extension. *Procedia Computer Science* **225**, 1415–1424 (2023)
- [23] Poveda-Villalón, M., Gómez-Pérez, A., Suárez-Figueroa, M.C.: Oops!(ontology pitfall scanner!): An on-line tool for ontology evaluation. *International Journal on Semantic Web and Information Systems (IJSWIS)* **10**(2), 7–34 (2014)
- [24] Saeedizade, M.J.: Large language models as assistants for ontology engineering. In: ISWC 2025 Companion Volume. CEUR-WS.org (2025), <https://ceur-ws.org/Vol-4085/paper18.pdf>
- [25] Saeedizade, M.J., Blomqvist, E.: Navigating ontology development with large language models. In: European Semantic Web Conference. pp. 143–161. Springer (2024)
- [26] Soares, F.M., Saraiva, A.M., Pires, L.F., Drucker, D.P., Braghetto, K.R., da Silva Santos, L.O.B., de Abreu Moreira, D., Corrêa, F.E., Delbem, A.C.B.: A novel ux-based approach for ontology evaluation: Applying tree testing to the agricultural product types ontology. *IEEE Access* (2025)
- [27] Svátek, V., Zamazal, O., Haniková, K., Chudán, D., Saeedizade, M.J., Blomqvist, E.: Welcome, new-born entity! on handling newly generated entities in ontology transformation. In: Companion Proceedings of the 24th International Conference on Knowledge Engineering and Knowledge Management, Amsterdam, Netherlands, CEUR Workshop Proceedings, To Appear. CEUR-WS. org (2024)
- [28] Tsaneva, S., Vasic, S., Sabou, M.: Llm-driven ontology evaluation: Verifying ontology restrictions with chatgpt. *The semantic web: ESWC satellite events* **2024** (2024)
- [29] Wu, G., Ling, C., Graetz, I., Zhao, L.: Ontology extension by online clustering with large language model agents. *Frontiers in Big Data* **7**, 1463543 (2024)